# 自研行情图表

## 自研行情图表

[原始文档 - Original Document] @ke.xu

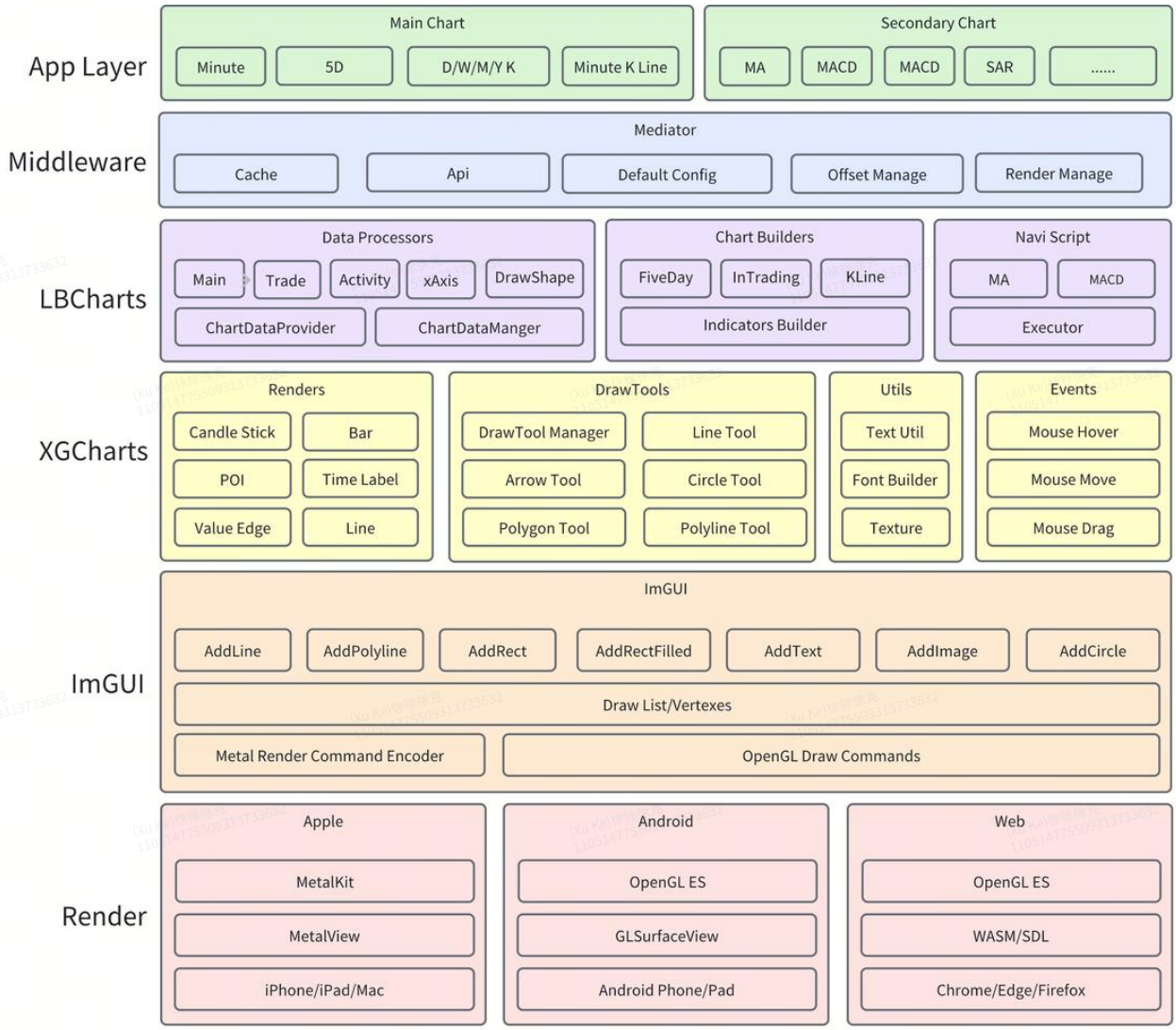
## 概述

自研行情技术分析图表，基于 C++ 编写，跨设备、跨平台支持，可运行在 iOS / Android / Desktop / Web 等多种设备和平台。

- 目前已经完成手机客户端、桌面端客户端替换，并已上线发布。
- 解决图表功能重复开发的问题，极大提高了图表相关业务开发效率。
- 功能多端统一，通过配置控制不同平台功能及特性。
- 丰富的内置功能：
  - 内置至少 60 种常用指标：KDJ、RSI、MACD...
  - 丰富的画线类型：空间尺、3 线段、黄金分割...

- 丰富的内置功能：股票叠加、筹码分布、买卖点、行动点...
- 内置多种主题，支持自定义组件配色。

- 相关链接：跨端图表 LBCharts




- APP Layer 配置图表类型、指标类型等。
- Middleware 获取当前数据，解析接口。
- LBCharts 解析数据，转换为图表所需绘制数据。
- XGCharts 解析绘制数据，转换为画笔数据。
- ImGUI 转换画笔数据到渲染接口数据。
- Render 完成绘制。

## 跨端、跨平台

一次开发，处处运行

一次开发，处处运行

- 基于 C++ 构建，可以运行在 iPhone、Android、Desktop、Web 等平台。
- 图表底层直接面向各个平台自有 GPU 渲染框架进行编程， 减少跨平台对图表本身带来的影响。https://github.com/ocornut/imgui
- 可通过 Bridge 接入原生代码，调用大部分原生 api 解决极端场景。时间格式化

## 基于 GPU 渲染构建

毫秒级渲染，无惧卡顿

毫秒级渲染，无惧卡顿

- 图表底层直接面向各个平台自有 GPU 渲染框架进行编程，从而获得最优性能表现，例如：
  - iOS 上使用 Metal Framework
  - Android 上使用 OpenGL ES
  - 并且这样做还可以获得更佳的平台兼容性，避免在不同平台上，由于平台与 GPU 渲染框架的沟通导致额外的性能损耗。
  - 在 Web 上，自研图表使用了 WebAssembly 来承载图表渲染运行时，可以获得比使用 JavaScript 更显著的性能优势。
- 经测试，目前 A14 芯片的 iPhone 12 上，单帧渲染可在 2~4ms 内完成，轻松应对高刷新率的挑战，实测在 iPhone 12 上可达 50 FPS 以上，iPhone 12 Pro Max 可达 80 FPS 以上（刷新率受限于 UIScrollView 的刷新回报速率以及 iOS 对屏幕刷新率的调度逻辑）。自研行情图表
- 受益于 GPU 绘制特性，对 CPU 的压力较小，因此自研图表即使在面对大周期范围内巨量 K 线的绘制场景，例如在一屏中渲染 5 年日 K 时，依然可以保持 50 FPS 以上的刷新率，用户操作没有丝毫卡顿。
- 此外 GPU 渲染能在低端设备上也具备优秀的体验表现，实测 iPhone 7 同样能提供 50 FPS 以上的刷新率。
- 基于可变刷新频率实现，分离数据处理和 GPU 渲染，极大降低了性能消耗。
- 渲染

## 集成 Navi 指标引擎

统一数据处理，拒绝数据碎片化

统一数据处理，拒绝数据碎片化

- 集成 Navi 指标引擎，可提供与国内常见技术指标平台（例如富途、通达信、文华）兼容的语法支持，并且提供全新的近编程语言的高阶语法支持，可实现更复杂的技术指标编写。

- 支持至少 60 种主流技术指标，可自定义编写。
- 支持复杂的行情指标编写，以及动态下发更多指标。
- 统一多种数据计算规则，提高数据处理开发效率，并且确保多端统一。

以成交量为例：`vol.nvs`

代码块

```
meta {
    title = "VOL",
    overlay = false
}

use quote;
let colors = close >= open ? (close > open ? #up : #666666) : (close < open ? #down : #666666);

plot(volume, title: "Volume", color: colors, style: plotstyle.histogram, format_number: true);
```

## 接入中间业务层

接管繁重业务，得心应手

接管繁重业务，得心应手

- 接入中间业务层，下沉原生侧业务逻辑，显著降低接入复杂度，通过缓存、API、 配置管理实现多端功能无缝同步、自由开关，目前功能内测中。
- 极大降低了接入成本，从以往的至少 2 周降低到 3 天左右，大大降低了接入门槛。
- 统一了代码实现逻辑，从以往的每端自由调用业务 api 到统一暴露通道 api，防止在特定情况下因业务代码调用顺序不同导致的逻辑问题。

```
<script>
  function kline() {
    Module.initializeConfig("desktop"); // 添加基础配置 key，映射为基础配置
    Module.setCounterID("ST/US/TSLA"); // 设置股票代码
    Module.setChartType(1, 1010); // 设置图表类型
  }
</script>
```

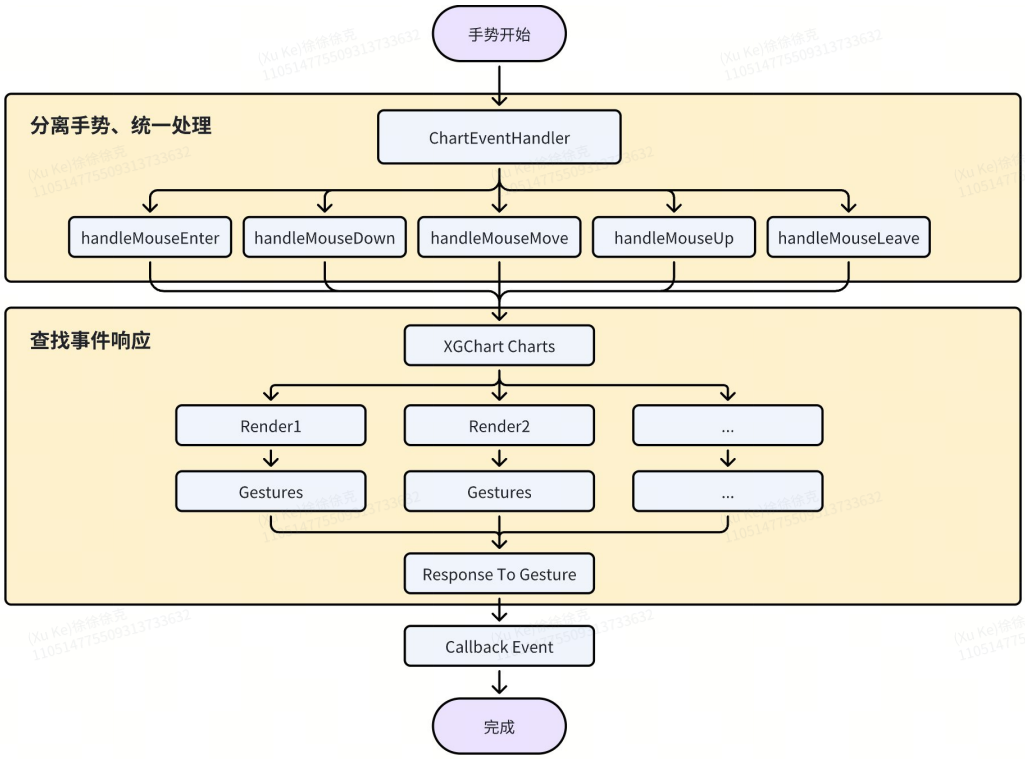
- 接管图表配置：`ChartsConfigDataModel` `ChartsQuoteConfigModel`

## 实现细节

## 手势实现

完整的事件传递链、响应链，目前已支持的手势：

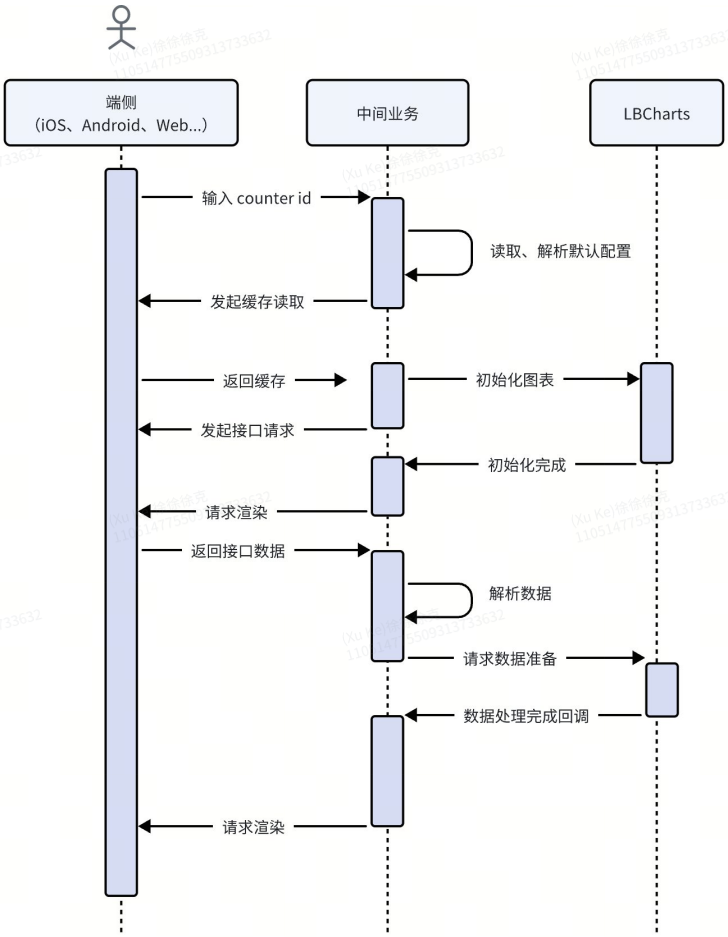
- 单击
- 双击
- 长按、拖动
- 缩放
- Hover
- 右键




Scroll

## 中间业务层实现

处理繁重的业务实现，避开业务代码调用，传入 counter_id 解决 90%的业务问题。



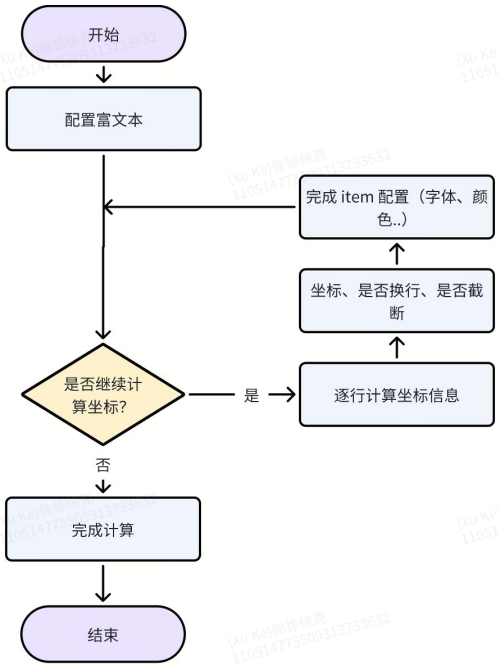

接入代码示例：

```
<script>
  function kline() {
    Module.initializeConfig("desktop"); // 添加基础配置 key，映射为基础配置
    Module.setCounterID("ST/US/TSLA"); // 设置股票代码
    Module.setChartType(1, 1010); // 设置图表类型
  }
</script>
```

实现不同平台不同特性

## 富文本实现

实现复杂文本场景，无惧文本设计。



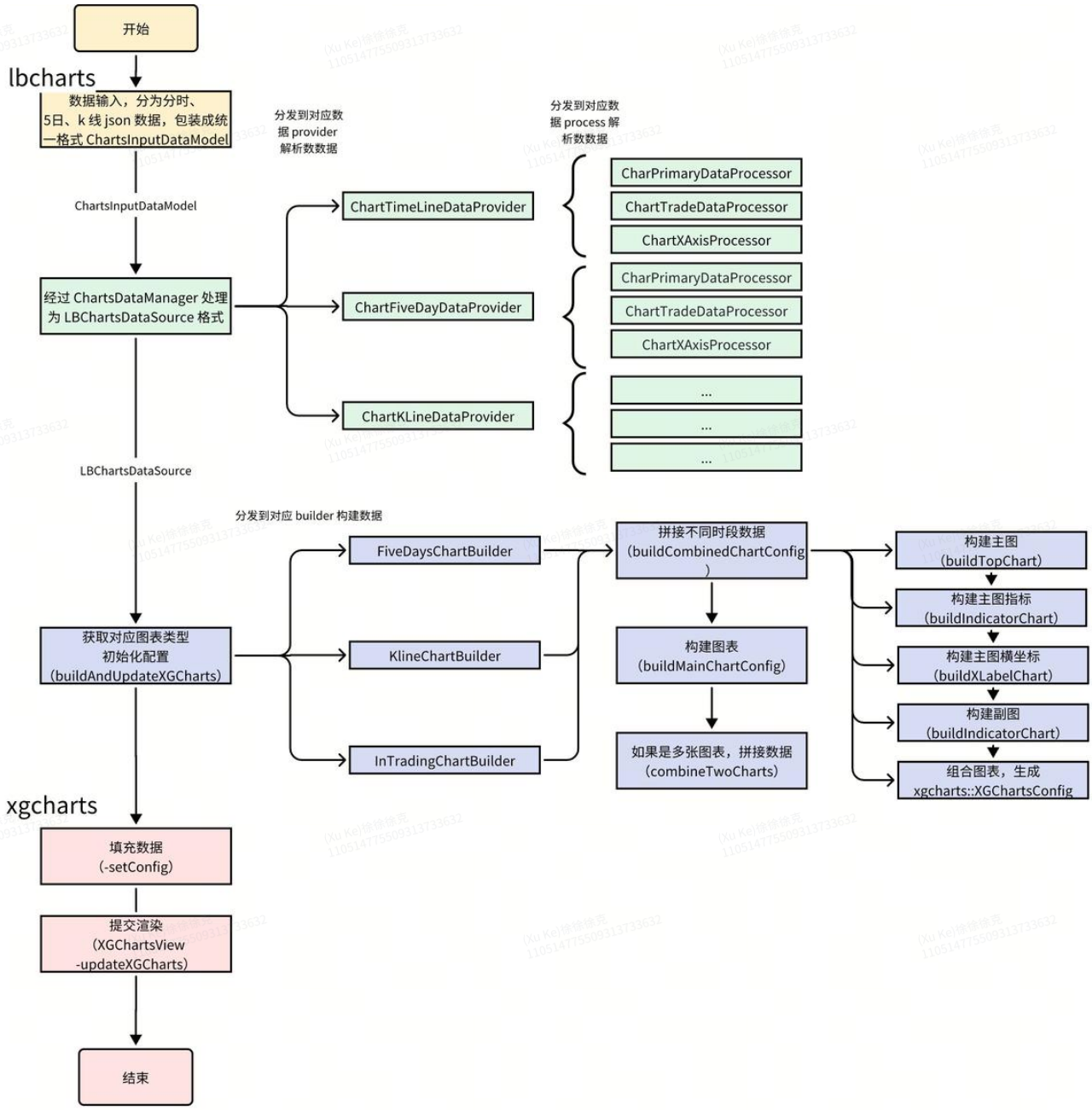

代码示例：

代码块

```
            AttributeTextGroup att_texts = {
                .frame = frame,
                .number_of_lines = 3,
                .line_break_clipping = true,
            };

            for in...
                att_texts.addText(AttributeText({color: "#ffaabb", font_size:12, "text": "文字1"}));
            ...

            att_texts.calculateFrames();

            att_texts.draw();
```

## 数据处理

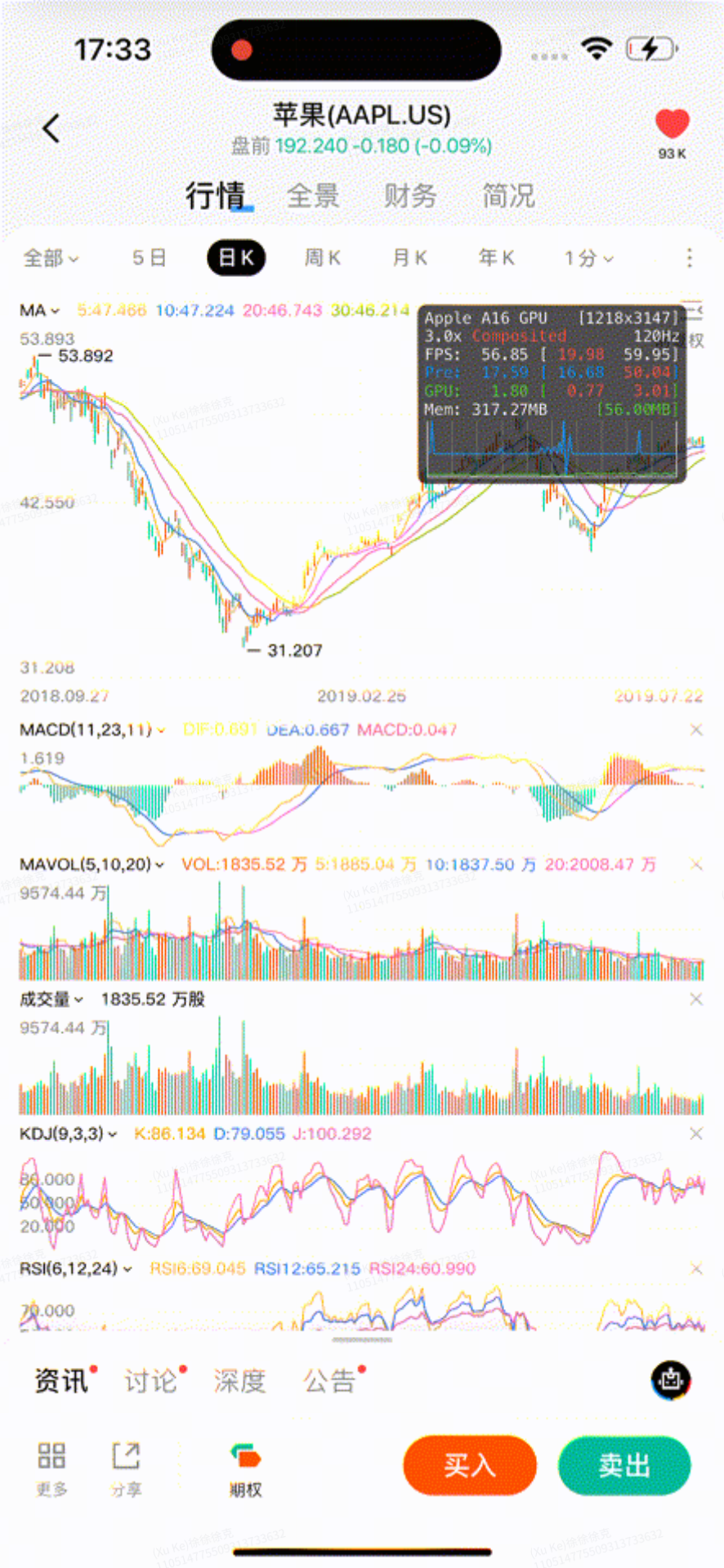
分发数据到对应处理单元， 模块化分责、单元自治。




## 友商对比视频

- 对比滑动操作，缩放操作

对比视频.mp4 (链接丢失)




## 未来计划

向 TradingView 看齐，完善功能同时降低接入成本。

- 优化手势，降低手势接入成本
- 增加画线类型、丰富图表功能
- 开放自定义指标编辑、用户自定义指标
- 降低数据处理耗时、增加图表数据兼容性、增加数据解析类型、降低初始化耗时
- 增加异常捕获能力，及时暴露潜在数据、渲染问题，完整的 log 功能
- 单独打包，完整的功能展示 DEMO